

개발자를 위한 클로드 코드와 커서 AI 실전

Claude Code와 Cursor AI를 활용한 개발 워크플로우 통합 인쇄본

강사: 생존코딩 오준석

Cursor

AI 모델의 원리와 코딩 에이전트 실전

- Part 1: AI의 기초 (AI Fundamentals) - 6 Chapters
- Part 2: 코딩 에이전트 실전 (Coding Agents) - 7 Chapters

강사: 생존코딩 오준석

전체 목차 (Overview)

Part 1. AI의 기초

1. AI 모델의 작동 원리
2. 환각(Hallucination)과 한계점
3. 토큰과 가격 책정 (Pricing)
4. 컨텍스트와 윈도우 (Context)
5. 도구 호출 (Tool Calling)
6. 에이전트의 원리 (Agents)

Part 2. 코딩 에이전트 실전

7. 에이전트 기본 활용법
8. 코드베이스 이해하기 (Indexing)
9. 기능 개발 워크플로우
10. 버그 탐색 및 자동 수정
11. 코드 리뷰와 테스트 자동화
12. 에이전트 커스터마이징 (Rules)
13. 종합 정리 및 베스트 프랙티스

Part 1. AI의 기초 (AI Fundamentals)

LLM의 내부 메커니즘부터 에이전트 아키텍처까지

1-1. AI 모델의 본질: 초고성능 범용 API

AI 모델은 만능 마법사가 아닌 "초고성능 범용 API 엔드포인트"입니다.

- 결제 처리를 위해 Stripe API를 호출하듯, 텍스트 분석·코드 생성을 위해 AI 모델을 호출합니다.

 결정론적(Deterministic) vs. 확률적(Probabilistic)

구분	전통적인 소프트웨어	AI / LLM 모델
동작 방식	결정론적 ($f(x) = y$)	확률적 ($P(w_t \mid w_{1:t-1})$)
재현성	동일한 입력 → 항상 동일한 출력	동일한 입력 → 매번 다른 경로/응답 가능
로직 제어	개발자가 모든 분기문을 명시적으로 작성	방대한 학습 가중치에 기반한 다음 단어 예측
멘탈 모델	"항상 정해진 규칙대로 실행된다"	"가장 그럴듯한 확률의 결과를 생성한다"

1-2. AI 모델의 예측 메커니즘과 선택 트레이드오프

AI 모델은 어떻게 다음 텍스트를 결정하는가?

1. 사전 학습된 데이터 (Training Data): 모델이 세상에 대해 배운 지식
2. 입력 프롬프트 (Prompt): 사용자가 현재 전달한 컨텍스트와 지시사항

핵심 시사점:

- 동일한 모델과 프롬프트라도 **매번 다른 답변**이 나올 수 있습니다.
- "한 단어로만 답해" 같은 엄격한 제약조건도 **확률적으로 무시될** 수 있습니다.

모델 선택의 3각 트레이드오프 (Trade-off)

- 지능 (Intelligence): 복잡한 아키텍처 설계, 깊은 추론 능력
- 속도 (Speed): 인라인 자동완성, 빠른 피드백 루프
- 비용 (Cost): 토큰당 과금 단가 및 API 유지 비용
 - 👉 작업 목적에 따라 Fast 모델(Sonnet/Flash)과 Heavy 추론 모델(o3/Opus)을 구분 사용

2-1. 환각(Hallucination)과 AI의 한계

환각은 버그가 아니라 확률적 텍스트 생성의 '본질적인 특성'입니다.

- LLM은 "참/거짓"을 판단하는 것이 아니라, "문맥상 다음에 올 가장 자연스러운 단어"를 생성합니다.
- 자신이 모른다는 사실을 인지하지 못하고, 매우 자신감 있는 어조로 그럴듯한 오답을 만듭니다.

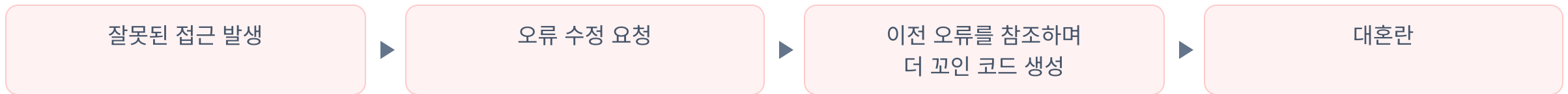
⚠ 대표적인 개발 환각 사례

- 존재하지 않는 패키지 / 메서드 날조: `import { magicalHelper } from 'react'`
- 폐기된(Deprecated) API 사용: 옛날 학습 데이터 기반의 과거 문법 생성
- 비일관된 타입 정의: 앞뒤가 맞지 않는 인터페이스 임의 생성

2-2. 앵커링 효과와 컨텍스트 오염 극복

⚓ 앵커링 효과 (Anchoring Effect)

- 대화 중간에 잘못된 코드나 가정이 한 번 컨텍스트에 포함되면, 모델은 그 잘못된 정보에 닻(Anchor)을 내리고 계속 오류를 증폭시킵니다.



💡 환각 & 앵커링 탈출 3원칙

- 즉시 새 세션 시작: 대화가 꼬이면 미련 없이 창을 닫고 새로 시작하기
- 단정적 제약보다 명확한 예시(Few-shot) 제공: "금지해"보다 올바른 형태 보여주기
- 컴파일러/린터 피드백 루프: AI의 출력을 항상 정적 분석 도구로 자동 검증

3-1. 토큰(Token)과 가격 책정 메커니즘

토큰(Token)이란?

- LLM이 텍스트를 처리하는 기본 단위 (단어 조각, Subword)
- 영문: 1 토큰 $\approx$ 약 4글자 (0.75 단어)
- 한국어/코드: 글자 수 대비 더 많은 토큰 소모 (UTF-8 인코딩 특성)

💰 입력 토큰 vs. 출력 토큰의 가격 차이

- 입력 토큰 (Prompt Tokens): 병렬 연산이 가능하여 상대적으로 저렴함
- 출력 토큰 (Completion Tokens): 직전 단어를 기반으로 1개씩 순차 생성되므로 3~5배 더 비싸고 느림

입력 (저렴 · 고속)

전체 컨텍스트 + 시스템 규칙 + 코드



LLM

토큰 단위 처리



출력 (고비용 · 순차)

생성된 코드

3-2. 프롬프트 캐싱 (Prompt Caching)과 비용 최적화

대규모 코드베이스에서의 비용 문제

- 질문할 때마다 전체 프로젝트 파일 목록, 룰셋(`.cursorrules`), 시스템 프롬프트를 전송하면 토큰 비용이 기하급수적으로 폭증합니다.

프롬프트 캐싱의 원리

- 모델 제공자(Anthropic, OpenAI 등)는 변하지 않는 앞부분 프롬프트(Prefix)를 메모리에 캐싱합니다.
- 캐시가 적중(Hit)하면 입력 비용 최대 90% 절감 + 응답 속도 수 배 향상!

개발자가 지켜야 할 캐싱 친화적 규칙

- 자주 바뀌지 않는 내용(규칙, 시스템 프롬프트)을 프롬프트 맨 앞에 배치
- 매 요청마다 프롬프트 앞단의 글자를 불필요하게 수정하지 않기

4-1. 컨텍스트(Context)와 컨텍스트 윈도우

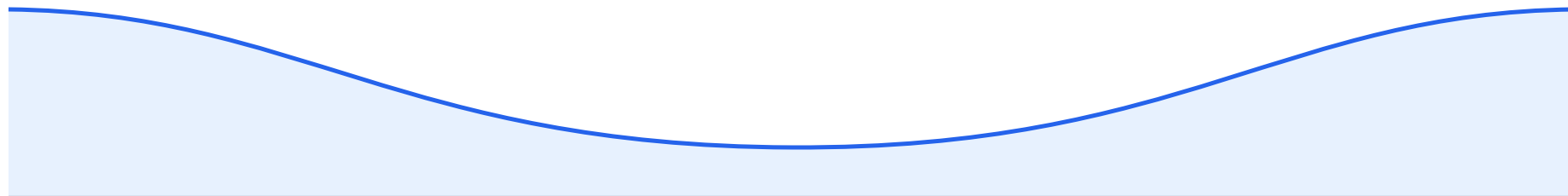
컨텍스트 윈도우 (Context Window)

- 한 번의 대화에서 모델이 동시에 '기억'하고 고려할 수 있는 최대 토큰 용량
- (예: 128k, 200k, 1M, 2M 토큰 등)

⚠ 'Lost in the Middle' (중간 망각 현상)

- 컨텍스트 윈도우가 아무리 커도, 모델의 주의력(Attention)은 프롬프트의 맨 앞(시작)과 맨 뒤(최근)에 집중됩니다.
- 방대한 코드의 중간에 묻힌 핵심 규칙이나 인터페이스는 무시되기 쉽습니다.

주의력(Attention)



프롬프트 시작

주의력 매우 높음

방대한 중간 데이터

주의력 급격히 저하 — 여기 묻힌 규칙은 무시됨

최근 질문 · 지시

주의력 매우 높음

4-2. 컨텍스트 품질 관리: Garbage In, Garbage Out

"양(Quantity)보다 관련성(Relevance)과 정밀함(Quality)이 핵심입니다."

- 전체 프로젝트를 무작정 다 밀어 넣으면 모델이 혼란에 빠집니다.

효과적인 컨텍스트 전달 전략

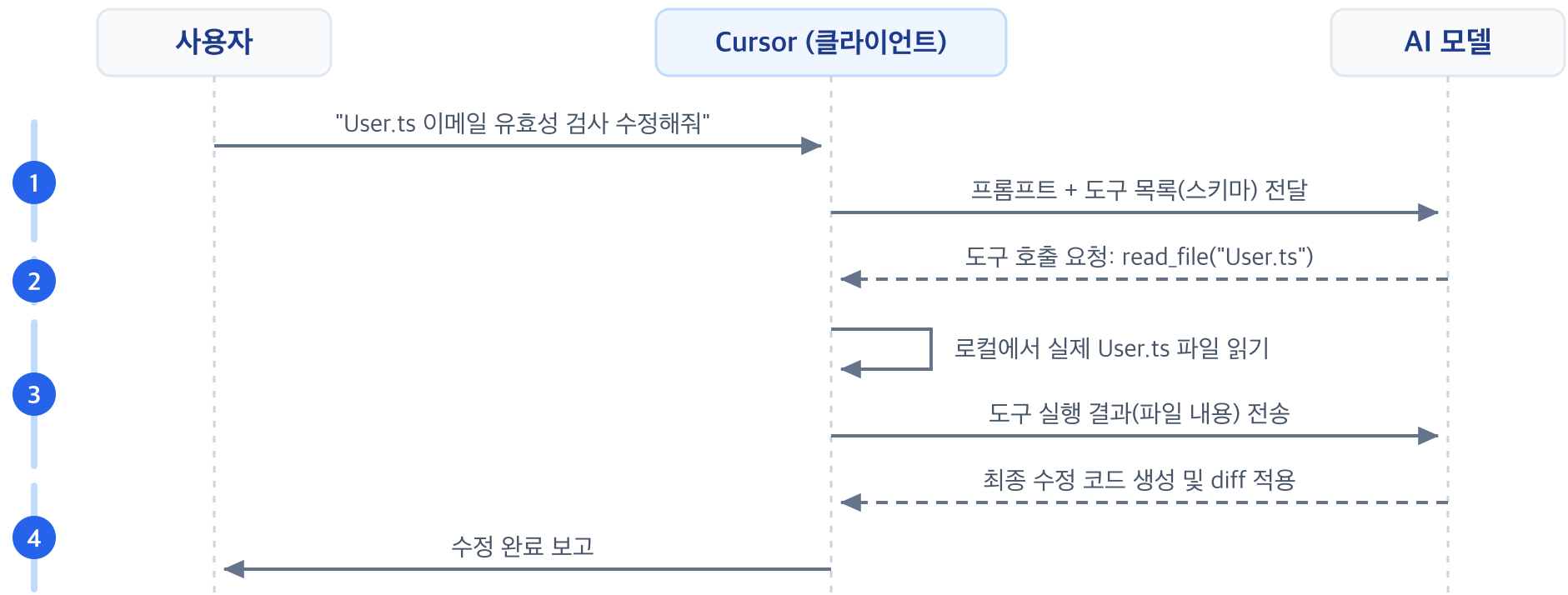
방식	설명	추천 상황
@Files / @Folders	특정 파일/폴더를 직접 지정	수정 대상과 연관 인터페이스가 명확할 때
@Codebase	임베딩 기반 시맨틱 검색으로 탐색	어떤 파일을 수정해야 할지 모를 때
@Docs	공식 외부 문서 라이브러리 참조	최신 라이브러리 API 문법이 필요할 때
.cursorignore	빌드 산출물, 로그, 거대 데이터 배제	토큰 낭비 및 노이즈 방지

5-1. 도구 호출 (Tool Calling / Function Calling)

LLM 자체는 외부 세상(파일 시스템, 인터넷, 터미널)과 직접 통신할 수 없습니다.

- 도구 호출(Tool Calling)은 LLM에게 '손과 발'을 달아주는 표준 인터페이스입니다.

도구 호출 4단계 메커니즘



5-2. 도구 정의(Schema)와 구조화된 출력

🔧 도구가 모델에 전달되는 형태 (JSON Schema)

```
{
  "name": "run_terminal_command",
  "description": "프로젝트 루트에서 셸 명령어를 실행합니다.",
  "parameters": {
    "type": "object",
    "properties": {
      "command": {
        "type": "string",
        "description": "실행할 명령어 (예: npm test)"
      }
    },
    "required": ["command"]
  }
}
```

- 모델은 실제 명령어를 실행하는 주체가 아닙니다.
- 단지 "어떤 도구를 어떤 인자값(Arguments)으로 실행해야 하는지" JSON으로 판단/출력하고, 실제 실행은 안전한 로컬 환경 (Cursor IDE)이 담당합니다.

6-1. AI 에이전트(Agent)의 원리: ReAct 루프

에이전트는 단순한 1회성 응답기가 아닌, 목표를 달성할 때까지 자율적으로 생각하고 도구를 사용하는 '추론 루프(Loop)' 시스템입니다.

🔄 ReAct (Reason + Act + Observe) 패턴



6-2. 단순 챗봇 vs. 코딩 에이전트 비교

항목	단순 대화형 AI (Chat)	자율 코딩 에이전트 (Agent)
상호작용	1회 질문 → 1회 답변	다단계 연속 작업 자율 수행
파일 조작	사용자에게 복사/붙여넣기 코드 제공	파일 직접 생성, 수정, 삭제, Diff 적용
환경 인지	정적 프롬프트에만 의존	터미널 실행, 린터 에러 감지, 웹 검색 연동
자가 치유	에러 발생 시 사용자가 복사해서 재질문	에러 발생 시 스스로 재시도 및 수정 (Self-Healing)
통제 방식	프롬프트 수정	중단(Stop), 승인(Approve), 체크포인트 롤백

Part 2. 코딩 에이전트 실전 (Coding Agents)

Cursor의 실전 워크플로우와 프로 개발자의 10x 생산성 기법

7-1. 에이전트와 효과적으로 협업하기 (Working with Agents)

에이전트는 '초급 개발자 팀원'과 같습니다. 명확한 목표와 경계선을 주어야 합니다.

🎯 에이전트 지시 3대 핵심 원칙

1. 명확한 목표(Goal) 설정:

- ✗ "로그인 기능 만들어줘"
- ○ "JWT 기반 Refresh Token 갱신 엔드포인트를 `/api/auth/refresh` 에 구현해줘"

2. 범위(Scope) 및 제약(Constraint) 명시:

- "기존 `UserRepository` 인터페이스를 깨뜨리지 말고 구현할 것"

3. 점진적 피드백(Iterative Feedback):

- 한 번에 거대한 요구를 하지 않고, 단계별로 커밋하며 진행

7-2. 세션 격리와 롤백 전략

세션 격리 (Fresh Session)

- 하나의 채팅 창에서 여러 작업을 연달아 수행하면 이전 작업의 토큰 찌꺼기로 인해 컨텍스트가 오염됩니다.
- 하나의 작업(기능 구현, 버그 수정 등)이 끝나면 새 채팅(`Cmd + L` / `Cmd + I`)을 열어 리셋하세요.

체크포인트와 롤백 (Checkpoint & Rollback)

- Cursor 에이전트는 파일 수정 전 체크포인트를 생성합니다.
- 에이전트가 엉뚱한 방향으로 코드를 대량 변경했다면, 주저하지 말고 'Reject All' 또는 체크포인트 복원을 사용하세요.

8-1. 코드베이스 이해하기: 색인과 검색 (Understanding Codebase)

Cursor는 내 수십만 줄의 프로젝트 코드를 어떻게 이해할까요?

- 정답: AST 기반 심볼 그래프(Symbol Graph) + 벡터 임베딩(Vector Indexing)의 결합

🔍 Cursor의 3단계 코드 색인 구조

1. 로컬 파일 파싱: 함수, 클래스, 인터페이스 간의 호출 관계(Call Graph) 분석
2. 청킹(Chunking) & 임베딩: 의미 단위로 쪼개어 고차원 벡터로 변환
3. 시맨틱 검색(Semantic Search): 질문의 의미와 가장 유사한 코드 조각을 즉각 추출

전체 프로젝트 코드



AST 분석 + 청킹



로컬 벡터 인덱스



질문과 가장 관련된
파일 주입

8-2. 코드베이스 탐색 심볼(@) 마스터하기

상황별 @ 멘션 선택 가이드

심볼	사용 목적	모범 사례 예시
@Codebase	프로젝트 전반의 흐름/구조 탐색	"우리 프로젝트에서 결제 완료 웹훅은 어디서 처리해?"
@Files	특정 파일 정밀 분석/수정	"@OrderService.ts 에 주문 취소 로직 추가해줘"
@Folders	특정 도메인 모듈 전체 참조	"@src/modules/auth 모듈의 아키텍처 구조 설명해줘"
@Docs	서드파티 라이브러리 공식 문서	"@Next.js App Router 15버전 Server Actions 문법으로 작성해줘"
@Git	최근 변경사항 / 커밋 내역 참조	"@Git 최근 커밋에서 발생한 변경점 요약해줘"

9-1. 새로운 기능 개발하기: Spec-First 워크플로우

초보자의 실수: 바로 에이전트에게 코드를 작성하라고 프롬프트 입력

👉 결과: 기존 아키텍처 무시, 엉뚱한 라이브러리 추가, 대규모 코드 충돌

📌 프로의 기능 개발 4단계 (Spec → Plan → Code → Verify)

1. 요구사항 명세화

Spec / RFC 작성

2. 단계별 구현 계획

Step-by-step Plan

3. 최소 단위 반복 구현

Small Iterations

4. 테스트 & 린트 검증

Verify & Commit

1. Ask 모드에서 계획 먼저 수립: "이 기능을 만들려고 해. 수정해야 할 파일과 계획을 먼저 짜줘"

2. 계획 검토 후 Agent 실행: 승인된 계획에 따라 파일 단위로 차례차례 구현

9-2. 에이전트와 함께하는 TDD (테스트 주도 개발)

AI 코딩에서 가장 강력한 안전장치는 바로 "테스트 코드"입니다.

- 에이전트에게 '정답의 기준(테스트)'을 먼저 주면, 에이전트가 스스로 테스트를 돌리며 완벽한 코드를 만들어냅니다.

Agent TDD 4단계 사이클

1. 인터페이스 정의: 함수 시그니처와 입력/출력 타입 결정
2. 실패하는 테스트 작성: `@Test` 에이전트에게 엡지 케이스 포함 테스트 코드 작성 지시
3. 구현 요청: "작성한 테스트가 모두 통과(Pass)하도록 비즈니스 로직을 구현해줘"
4. 자가 검증(Self-Verification): 에이전트가 `npm test` 를 실행하며 통과할 때까지 코드 자동 수정

10-1. 버그 탐색 및 수정 (Finding & Fixing Bugs)

🐛 버그 수정 시 에이전트에게 주어야 할 3가지 황금 컨텍스트

1. 정확한 에러 메시지와 스택 트레이스 (Stack Trace)
2. 버그가 발생하는 재현 경로 (Reproduction Steps)
3. 기대했던 결과(Expected) vs 실제 발생한 결과(Actual)

❌ 나쁜 프롬프트: "결제창에서 에러 나는데 고쳐줘"

○ 좋은 프롬프트:

"장바구니에서 쿠폰 적용 후 결제 버튼을 누르면 아래와 같은 500 에러가 발생해.

[에러 로그 붙여넣기]

@PaymentService.ts 의 할인율 계산 로직에서 null 처리가 누락된 것 같아. 원인을 분석하고 수정해줘."

10-2. 터미널 자가 치유 (Self-Healing Debugging)

Cursor는 터미널 출력(Terminal Output)을 직접 인식하고 분석합니다.

터미널 에러 해결 흐름

- 터미널에서 컴파일 에러 / 테스트 실패 발생 시:
 - i. 터미널 상단의 **Add to Chat** 또는 단축키 클릭
 - ii. 에이전트가 에러 로그를 읽고 관련된 소스 코드를 자동으로 추적
 - iii. 근본 원인(Root Cause)을 찾아 수정 패치를 제시

근본 원인 vs 임시 땀질 코드 판별법

- 에이전트가 단순히 `try-catch` 로 에러를 덮거나 `any` 타입으로 때우려 하는지 diff에서 반드시 확인하세요!

11-1. 코드 리뷰와 테스트: Diff 검증의 기술

"에이전트가 만든 코드는 '인턴이 작성한 PR'을 리뷰하듯 꼼꼼히 확인해야 합니다."

- 에이전트가 제안한 Diff를 'Accept'하기 전에 확인해야 할 체크리스트입니다.

🔍 Diff 리뷰 5대 체크리스트

1. 불필요한 코드 삭제 유무: 기존의 유효한 로직이 주석 처리되거나 삭제되지 않았는가?
2. 안티패턴 및 보안 취약점: 하드코딩된 시크릿 키, SQL 인젝션, 미검증 사용자 입력이 없는가?
3. 타입 안전성: `as any`, `unknown` 남발로 타입스크립트의 안전망을 훼손하지 않았는가?
4. 패키지 의존성 오염: 쓰지도 않는 엉뚱한 외부 라이브러리를 `package.json`에 추가하지 않았는가?
5. 컨벤션 일관성: 기존 프로젝트의 네이밍 규칙과 포맷을 따르고 있는가?

11-2. 자동화된 회귀 테스트 생성

🛡 버그를 잡은 직후 반드시 수행해야 할 작업

"다시는 같은 버그가 재발하지 않도록 회귀 테스트(Regression Test) 추가하기"

프롬프트 템플릿 예시

방금 수정한 [결제 금액 음수 입력 버그]에 대해:

1. 정상 결제 케이스
2. 0원 결제 케이스
3. 음수 금액 전달 시 `CustomValidationException`을 던지는 엡지 케이스
이 3가지를 검증하는 단위 테스트를 `@PaymentService.spec.ts`에 추가해줘.

👉 버그 수정과 테스트 작성을 한 세트로 묶는 습관이 견고한 소프트웨어를 만듭니다.

12-1. 에이전트 커스터마이징: `.cursorrules` & MDC

프로젝트의 아키텍처 규칙과 코딩 컨벤션을 Cursor의 두뇌에 영구 주입하는 방법

- Cursor는 작업할 때마다 이 규칙들을 시스템 프롬프트의 최상단에 자동으로 로드합니다.

규칙 파일 구성 체계

방식	위치	특징 및 용도
레거시 단일 규칙	<code>.cursorrules</code>	프로젝트 루트에 위치하는 단일 마크다운/텍스트 파일
모듈형 규칙 (MDC)	<code>.cursor/rules/*.mdc</code>	파일 패턴별(glob)로 특정 파일 수정 시에만 활성화되는 고도화된 규칙 시스템

12-2. 실전 `.cursor/rules` 작성 예시

```
---  
description: React 컴포넌트 작성 시 필수 코딩 표준  
globs: src/components/**/*.{tsx, jsx}  
---
```

React 개발 규칙

- 모든 컴포넌트는 함수형 컴포넌트(Functional Component)로 작성한다.
- 스타일링은 반드시 Tailwind CSS 유틸리티 클래스를 사용하며 인라인 style은 금지한다.
- 상태 관리는 Zustand를 기본으로 사용한다.
- 비동기 데이터 패칭에는 TanStack Query(React Query)의 커스텀 훅을 활용한다.
- 모든 props는 TypeScript 인터페이스로 명시적 타입을 정의한다.

규칙 작성 팁: "절대 ~하지 마라" 같은 부정형 지시보다는 "~할 때는 반드시 A 패턴을 사용하라"는 긍정형/예시 중심 지시가 훨씬 잘 작동합니다.

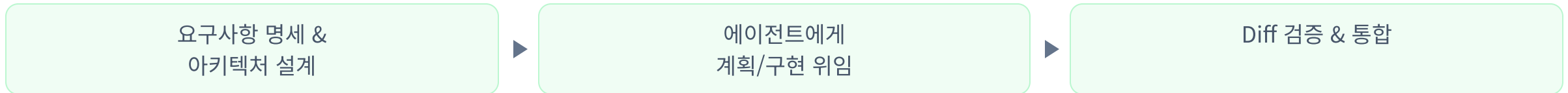
13-1. 종합 정리: 개발자의 새로운 멘탈 모델

AI 시대, 개발자의 역할은 '타이피스트(Typist)'에서 '설계자이자 오케스트레이터(Architect & Reviewer)'로 진화했습니다.

과거의 개발 방식 — 물리적 타이핑 80%



현대의 AI 에이전트 페어 프로그래밍 — 사고 & 검증 80%



- **더 중요한 역량:** 문제 정의 능력, 시스템 아키텍처 이해도, 엄격한 코드 리뷰 감각
- **AI에게 위임할 영역:** 반복적인 보일러플레이트 작성, 문법 변환, 기초 테스트 코드 생성

13-2. Cursor 200% 활용을 위한 10대 계명

1. AI는 마법이 아니라 확률적 API임을 항상 인지하라.
2. 대화가 꼬이거나 다른 작업을 시작할 땐 즉시 새 세션(`Cmd+L`)을 열어라.
3. 무작정 코드부터 짜게 하지 말고, Spec과 계획(Plan)을 먼저 합의하라.
4. 전체 파일을 다 주지 말고 `@Files` 로 가장 핵심적인 컨텍스트만 정밀하게 전달하라.
5. `.cursorrules` 와 `.cursorignore` 로 프로젝트 컨벤션을 자동화하라.
6. 에이전트에게 테스트(TDD)라는 채점 기준표를 쥐어주어라.
7. 에러를 만나면 스택 트레이스와 기대 결과를 정확히 전달하라.
8. 제안된 Diff를 맹신하지 말고 인턴의 PR처럼 꼼꼼히 리뷰하라.
9. 도구 호출과 터미널 연동을 적극 활용하여 피드백 루프를 단축하라.
10. 개발의 최종 책임은 항상 인간 개발자에게 있음을 잊지 마라.

Q & A

감사합니다!

생존코딩 오준석

Claude Code

터미널에서 실행하는 차세대 에이전틱 코딩 마스터

- **Part 1:** 기초 및 기본기 (Foundations & Setup) - 4 Chapters
- **Part 2:** 실전 워크플로우 (Workflows & Context) - 4 Chapters
- **Part 3:** 심화 에이전틱 확장 (Advanced Agentic Features) - 4 Chapters
- **Part 4:** 종합 마무리 (Wrap-up & Quiz) - 1 Chapter

강사: 생존코딩 오준석

전체 목차 (Overview)

Part 1. 기초 및 기본기

1. Claude Code란 무엇인가?
2. Claude Code의 동작 원리
3. 설치 및 환경 설정
4. 첫 번째 프롬프트 작성

Part 2. 실전 워크플로우

5. EPCC 표준 워크플로우
6. 컨텍스트 관리 전략
7. 코드 리뷰 워크플로우
8. CLAUDE.md 파일 활용

Part 3. 심화 에이전틱 확장

9. 서브에이전트 (Subagents)
10. 스킬 (Skills) 확장
11. Model Context Protocol (MCP)
12. 가드레일 (Hooks)

Part 4. 종합 마무리

13. 종합 요약 & 실전 자가진단 퀴즈

Part 1. 기초 및 기본기 (Foundations & Setup)

AI 코딩 어시스턴트에서 자율형 에이전트로의 패러다임 전환

Module 01: Claude Code란 무엇인가?

💡 패러다임의 변화: Chatbot vs AI Agent

- 기존 Chatbot: 복사-붙여넣기 기반, 코드 조각 제안, IDE 외부 격리 환경
- Claude Code (Agent): 로컬 터미널/파일시스템에 직접 접근하여 파일 읽기/수정, 명령어 실행, 테스트 검증까지 자율 수행

구분	일반 AI 챗봇 / 인라인 완성	Claude Code (Agentic CLI)
작동 환경	웹 브라우저 / 에디터 단일 라인	로컬 터미널 & 전체 워크스페이스
컨텍스트 이해	복사된 일부 코드 조각	전체 프로젝트 구조 및 파일 관계
작업 실행	사용자가 수동 복사 & 실행	직접 파일 생성/수정, Bash 명령 실행
검증 방식	사용자가 수동 빌드 & 오류 확인	스스로 테스트/빌드 실행 후 자가 수정

Module 01: Claude Code의 핵심 특징

⚡ 왜 터미널(CLI) 기반인가?

1. 개발자 친화적 워크플로우

- 터미널, Git, 빌드 도구, 패키지 매니저와의 완벽한 융합
- IDE 종류(VS Code, Cursor, JetBrains, Vim 등)에 종속되지 않음

2. 풀 스택 자율성

- 단순 코드 작성을 넘어 파일 생성, 패키지 설치, DB 마이그레이션, Git 커밋까지 일괄 처리

3. 인간 중심의 통제권 (Human-in-the-Loop)

- 위험한 Bash 명령어 및 파일 수정 시 사용자의 명시적 승인(`y/n`) 절차 제공

생존코딩 Pro-Tip: Claude Code는 개발자를 대체하는 것이 아니라, 명령을 완벽하게 수행하는 최상급 시니어 부사수를 터미널에 상주시키는 것입니다.

Module 02: Claude Code의 동작 원리

🔄 에이전틱 루프 (The Agentic Loop)

Claude Code는 단순히 한 번의 프롬프트에 답변하는 것이 아니라, 목표가 완수될 때까지 **4단계 루프**를 반복합니다.



Module 02: 도구(Tools)와 권한 모델

🔧 Claude Code가 사용하는 핵심 내장 도구

- `View / Read` : 파일 내용 확인 및 라인 단위 조회
- `Edit / Replace` : 파일의 특정 블록을 정밀하게 수정 (diff 기반)
- `Write / Create` : 신규 파일 생성 및 덮어쓰기
- `Bash` : 터미널 명령어(테스트, 빌드, git, npm 등) 실행
- `Glob / Grep` : 파일 탐색 및 코드베이스 패턴 검색

권한 모델과 안전성:

- 읽기/검색 작업: 기본 자동 허용 (빠른 탐색 지원)
- 파일 수정/Bash 실행: 실행 전 **Diff 및 명령어 미리보기** 제공 후 사용자 승인 요청
- 신뢰 모드(`--dangerously-skip-permissions`)는 격리된 컨테이너 환경에서만 권장

Module 03: 설치 및 환경 설정

⚙️ 시스템 요구사항 및 전역 설치

Claude Code는 Node.js 기반의 글로벌 CLI 도구로 제공됩니다.

```
# 1. Node.js 18 이상 버전 확인
node -v

# 2. Claude Code 글로벌 설치
npm install -g @anthropic-ai/claude-code

# 3. 설치 확인 및 버전 점검
claude --version
```

🔑 인증(Authentication) 2가지 방식

1. Claude Web 계정 연동 (권장): 브라우저를 통한 OAuth 로그인

2. Anthropic Console API Key: `export ANTHROPIC_API_KEY="sk-ant-..."`

Module 03: 작업 환경 및 IDE 통합

터미널 및 에디터 연동 세팅

터미널에서 직접 실행

```
# 프로젝트 루트 디렉토리로 이동
cd /path/to/my-project

# Claude 세션 시작
claude
```

단축 실행 (One-shot Mode)

```
# 특정 명령을 바로 실행하고 종료
claude -p "모든 단위 테스트를 실행하고 실패 원인을 분석해줘"
```

VS Code / JetBrains 내장 터미널

- 단축키: `Ctrl + `` (터미널 토글)
- 에디터 우측/하단에 터미널을 열고 `claude` 를 실행하면 코드 실시간 변경 확인 용이

주요 인터랙티브 단축키

- `Ctrl + C` : 현재 실행 중인 에이전트 작업 중단
- `Esc + Enter` : 멀티라인 프롬프트 입력 모드
- `Tab` : 명령어 및 파일명 자동완성

Module 04: 첫 번째 프롬프트 작성

🎯 성공적인 에이전트 프롬프트 3원칙

1. **명확한 목표(Goal):** 무엇을 만들거나 고쳐야 하는지 구체적으로 서술
2. **제약 조건(Constraints):** 기술 스택, 라이브러리 버전, 수정 금지 파일 등 명시
3. **검증 기준(Verification):** 작업 성공 여부를 어떻게 판별할지 제시 (예: 테스트 통과)

❌ 나쁜 프롬프트

```
> 로그인 화면 만들어줘.  
> 버그 좀 고쳐봐.
```

모호하여 불필요한 파일 탐색과 잘못된 추측 유발

✅ 좋은 프롬프트

```
> src/auth/ 에 JWT 기반 로그인 API를 작성해줘.  
> - bcrypt 비밀번호 해싱 적용  
> - 기존 User 모델(src/models/user.ts) 활용  
> - 완료 후 npm test auth 실행해서 검증할 것
```

Module 04: 대화형 피드백 루프 실습

💬 인터랙티브 피드백 사이클

User: "게시글 목록 조회 API를 페이지네이션 기능과 함께 구현해줘."

↓

Claude: "코드베이스를 탐색하여 기존 Post 모델과 라우터를 확인했습니다.
Limit/Offset 방식으로 구현할까요, 아니면 Cursor 기반으로 할까요?"

↓

User: "실시간 데이터가 많으니 Cursor 기반으로 구현해줘."

↓

Claude: "Cursor 기반 페이지네이션 구현 및 단위 테스트 작성을 완료했습니다.
diff를 확인하시고 승인해 주세요."

생존코딩 Pro-Tip: 한 번에 너무 거대한 요구사항을 던지기보다, 1~2개 단위의 세부 작업으로 나누어 에이전트와 대화하며 점진적으로 기능을 완성하는 것이 훨씬 정확합니다.

Part 2. 실전 워크플로우 (Workflows & Context)

체계적인 개발 프로세스와 컨텍스트 최적화

Module 05: EPCC 표준 워크플로우

🏆 Anthropic 공식 추천 4단계 개발 사이클

대규모 프로젝트에서 실패 없이 작업을 완료하기 위한 표준 프로세스입니다.



1. **Explore (탐색)**: 코드를 수정하지 않고 관련 파일, 의존성, 아키텍처를 먼저 분석
2. **Plan (계획)**: 변경할 파일 목록, 단계별 로직, 롤백 전략을 사전 설계
3. **Code (구현 & 검증)**: 계획에 따라 최소 단위로 수정하고 즉시 빌드/테스트 수행
4. **Commit (커밋)**: Git Diff를 최종 점검하고 커밋 메시지 작성 및 푸시

Module 05: EPCC 단계별 실전 프롬프트

실제 터미널 프롬프트 예시

1. Explore & Plan

```
> [Explore] 결제 모듈 리팩토링을 하려고 해.  
> 결제 관련 파일들을 모두 찾아서 구조와  
> 의존성 다이어그램을 요약해줘.  
> (아직 코드는 수정하지 마!)  
  
> [Plan] 토스페이먼츠 연동을 위한  
> 단계별 구현 계획서를 작성해줘.
```

2. Code & Commit

```
> [Code] 계획서의 1단계(Webhook 처리)를  
> 구현하고 npm test payment로 검증해줘.  
  
> [Commit] git status와 diff를 확인하고,  
> Conventional Commits 규칙에 맞추어  
> 커밋 메시지를 작성하고 커밋해줘.
```

규칙: 복잡한 작업일수록 "먼저 분석만 하고 계획을 세워줘"라고 요청하여 탐색과 코딩을 분리하세요!

Module 06: 컨텍스트 관리 전략

Context Window와 Context Defocus

- LLM은 대화가 길어질수록 컨텍스트 윈도우가 가득 차고, **토큰 비용 증가** 및 **성능 저하(Defocus)**가 발생합니다.

필수 컨텍스트 관리 슬래시 명령어

- `/compact` : 현재까지의 긴 대화 내역을 핵심 요약본으로 압축하여 토큰 공간 확보
- `/clear` : 대화 기록을 완전히 초기화 (새로운 독립 작업을 시작할 때 필수)
- `/cost` : 현재 세션에서 사용한 토큰 양과 누적 비용 확인
- `/context` : 현재 컨텍스트 윈도우에 로드된 파일 및 메모리 점유율 확인

Module 06: 컨텍스트 최적화 베스트 프랙티스

❌ 피해야 할 패턴

- 하나의 긴 세션에서 전혀 다른 3~4개 기능을 연속으로 작업하기
- 대용량 로그 파일이나 minified 빌드 산출물을 통째로 읽게 하기
- 오류 발생 시 `/compact` 나 리셋 없이 무한 반복 질문하기

✅ 권장 베스트 프랙티스

- 작업 단위(기능 구현, 버그 수정)가 끝날 때마다 `/clear` 또는 새 세션 시작
- 대용량 파일은 필요한 줄 번호 범위(`view_file StartLine/EndLine`)로 제한 조회
- 중요한 프로젝트 규칙은 `CLAUDE.md` 에 작성하여 세션이 초기화되어도 유지되도록 구성

Module 07: 코드 리뷰 워크플로우

🔍 에이전트를 활용한 3단계 코드 리뷰

Claude Code는 단순 작성을 넘어 강력한 **코드 리뷰어** 역할을 수행합니다.

1. 변경점 추출

Git Diff / PR



2. 다각도 분석

보안 · 성능 · 컨벤션



3. 개선안 제시

자동 수정까지 수행

📌 주요 리뷰 관점

1. 보안 취약점: SQL Injection, XSS, 민감정보 하드코딩 여부 점검
2. 에지 케이스 & 오류 처리: null/undefined 예외, 비동기 Promise 에러 핸들링
3. 프로젝트 컨벤션 준수: 네이밍 규칙, 폴더 구조, 타입스크립트 strict 모드

터미널에서 코드 리뷰 요청

```
claude -p "현재 브랜치와 main 브랜치 간의 git diff를 분석하여 코드 리뷰 리포트를 작성해줘."
```

Module 07: Human-in-the-Loop 코드 리뷰

🛡️ 사람이 통제권을 유지하는 리뷰 원칙

- 에이전트 제안을 무조건 신뢰하지 말 것: AI는 유효해 보이지만 미묘한 로직 버그를 만들 수 있음
- **Interactive Diff 확인**: Claude Code가 파일을 변경하기 전에 보여주는 diff를 직접 눈으로 확인
- **반려 및 재지시**: 마음에 들지 않는 변경점은 `n` 으로 거부하고, 구체적인 수정 이유를 피드백

Claude: "src/api/user.ts 파일의 42~58 라인을 수정하겠습니다. 승인하시겠습니까? (y/n)"

User: "n, DB 쿼리를 직접 날리지 말고 기존 UserRepository의 findById 메서드를 재사용해줘."

Claude: "UserRepository.findById를 재사용하도록 수정했습니다. 다시 확인해 주세요."

Module 08: CLAUDE.md 프로젝트 가이드

프로젝트의 영구 메모리 **CLAUDE.md**

CLAUDE.md 는 프로젝트 루트에 위치하는 마크다운 파일로, 세션이 시작될 때마다 Claude가 가장 먼저 읽는 불변의 프로젝트 지침서입니다.

CLAUDE.md에 반드시 포함해야 할 핵심 4요소

1. **Tech Stack & Architecture:** 프레임워크, 언어 버전, 주요 아키텍처 패턴
2. **Build & Test Commands:** `npm run build` , `pytest` , `npm test` 등 빌드/테스트 명령어
3. **Coding Standards:** 네이밍 컨벤션, 코드 스타일, 금지된 라이브러리
4. **Workflow Rules:** "테스트 코드 필수 작성", "커밋 전 linter 실행" 등 작업 수칙

Module 08: 실전 CLAUDE.md 템플릿

프로젝트 개발 지침 (CLAUDE.md)

1. 빌드 및 테스트 명령어

- 개발 서버 실행: ``npm run dev``
- 테스트 실행: ``npm test`` (특정 파일: ``npm test -- <path>``)
- 린트 검사: ``npm run lint``

2. 코딩 컨벤션 & 아키텍처

- TypeScript Strict 모드 준수 (any 타입 사용 금지)
- 상태 관리는 Zustand를 사용하며, 컴포넌트는 Functional + Hooks 구조 유지
- API 통신 시 반드시 ``/src/api/client.ts``의 공통 인스턴스 사용

3. 작업 수칙 (Crucial Rules)

- 코드 수정 후에는 반드시 관련된 단위 테스트를 실행하여 통과 확인
- 전체 파일을 덮어쓰지 말고 diff 기반으로 수정할 것
- 한국어로 커뮤니케이션하고, 커밋 메시지는 Conventional Commits 영문 작성

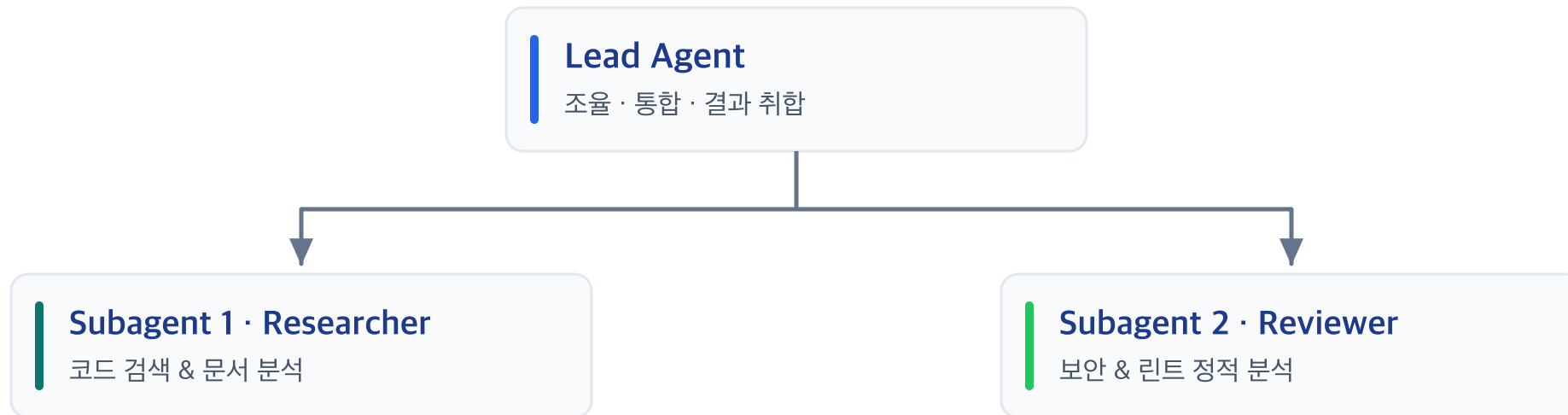
Part 3. 심화 에이전틱 확장 (Advanced Agentic Features)

Subagents, Skills, MCP, Hooks로 완성하는 엔터프라이즈 자동화

Module 09: 서브에이전트 (Subagents)

👤 단일 에이전트의 한계와 서브에이전트 분할

- **문제점:** 하나의 에이전트가 리서치, 아키텍처 설계, 코딩, 테스트, 문서화를 모두 수행하면 컨텍스트 윈도우가 급격히 오염됨
- **해결책 (Subagents):** 독립된 컨텍스트를 가진 보조 에이전트를 생성하여 특정 작업을 위임하고 결과만 메인 세션에 반환



각 서브에이전트는 독립된 컨텍스트에서 작업하고, 결과만 메인 세션으로 반환합니다

Module 09: 서브에이전트 활용 패턴

실전 서브에이전트 위임 시나리오

1. 대규모 코드베이스 사전 조사 (Research Subagent)

- 메인 에이전트의 토큰을 아끼기 위해 수십 개의 파일을 조사하는 역할을 서브에이전트에 위임
- 결과 요약 보고서만 메인 에이전트에 전달

2. 격리된 테스트 작성 및 실행 (Tester Subagent)

- 작성된 코드에 대한 엡지 케이스 테스트 생성을 서브에이전트에 독립 위임

3. 독립된 워크스페이스 분기

- 브랜치/워크트리를 분리하여 메인 코드에 영향을 주지 않고 안전한 프로토타이핑 수행

효과: 메인 세션의 컨텍스트를 깨끗하게 유지하여, 수백 번의 턴이 이어지는 대형 프로젝트도 지능 저하 없이 완수 가능!

Module 10: 커스텀 스킬 (Skills)

⚡ Skills: 온디맨드(On-Demand) 지식 확장

- `CLAUDE.md` : 모든 세션에 항상 로드됨 (글로벌 규칙, 과도하면 토큰 낭비)
- **Skills:** `.claude/skills/` 디렉토리에 정의되며, **필요할 때만 호출**되어 토큰을 획기적으로 절약

스킬 디렉토리 구조

```
.claude/  
└─ skills/  
    ├── deploy/  
    │   └─ SKILL.md  
    ├── db-migrate/  
    │   └─ SKILL.md  
    └─ code-review/  
        └─ SKILL.md
```

동작 방식

- 사용자가 `/deploy` 명령 입력 또는
- Claude가 배포 작업임을 감지했을 때만 `SKILL.md` 지침을 컨텍스트에 주입
- 작업 완료 후 컨텍스트에서 제외

Module 10: 실전 SKILL.md 작성 예시

🔧 배포 자동화 스킬 (`.claude/skills/deploy/SKILL.md`)

```
---
name: deploy
description: 스테이징 또는 프로덕션 환경에 애플리케이션을 안전하게 배포합니다.
---

# 배포 절차 지침

1. 사전 검증:
  - `npm run lint` 및 `npm test`가 통과하는지 먼저 확인하라.
  - 미커밋된 Git 변경사항이 있는지 확인하라.

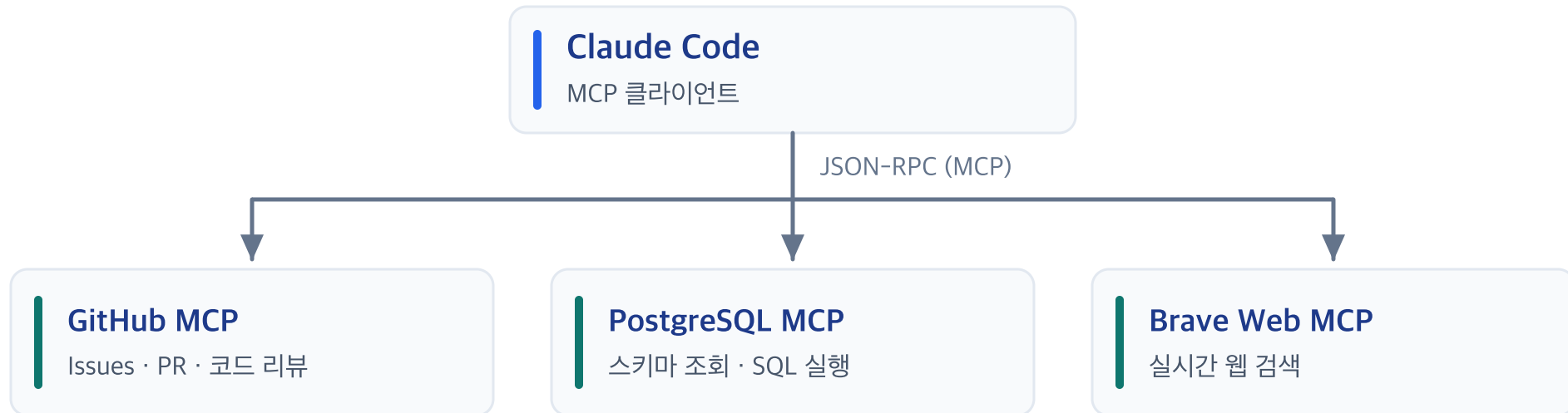
2. 빌드 실행:
  - `npm run build`를 실행하고 빌드 아티팩트 생성을 확인하라.

3. 환경별 배포 명령어:
  - Staging: `npx wrangler pages deploy dist --project-name=my-app-staging`
  - Production: 사용자의 명시적 2차 확인을 받은 후 배포 실행
```

Module 11: Model Context Protocol (MCP)

🌐 MCP(Model Context Protocol)란?

- Anthropic이 주도하는 AI와 외부 도구/데이터 간의 오픈 표준 통신 프로토콜
- Claude Code가 로컬 파일시스템을 넘어 **GitHub, DB, Slack, Jira, 웹 브라우저** 등과 직접 통신



Module 11: MCP 서버 연동 및 실전 활용

🔌 MCP 연동 명령어 및 활용 사례

```
# 1. PostgreSQL 데이터베이스 MCP 서버 추가
claude mcp add postgresql npx -y @modelcontextprotocol/server-postgres "postgresql://user:pass@localhost:5432/mydb"

# 2. GitHub MCP 서버 추가 (PR/Issue 직접 제어)
claude mcp add github npx -y @modelcontextprotocol/server-github

# 3. 등록된 MCP 목록 확인
claude mcp list
```

💡 실전 활용 프롬프트

- > "DB 스키마를 조회해서 users 테이블과 orders 테이블의 관계를 바탕으로 통계 API를 만들어줘."
- > "GitHub Issue #142번을 읽고 해당 버그를 수정한 뒤 PR을 올려줘."

Module 12: 가드레일 (Hooks)

🔗 Hooks: 결정론적(Deterministic) 통제 레이어

- 프롬프트의 한계: "파일 수정 후 항상 linter를 돌려줘"라고 프롬프트로 지시하면 100% 보장하기 어려움
- Hooks의 위력: 이벤트 생명주기에 훅(Hook)을 걸어 강제적이고 확실하게(Deterministic) 스크립트를 실행

🕒 핵심 생명주기 이벤트 (Lifecycle Events)

- `SessionStart` : 세션 시작 시 환경 점검 및 초기 설정 실행
- `PreToolUse` : 도구 실행 직전 가로채기 (예: 위험한 `rm -rf`, `DROP TABLE` 명령어 차단)
- `PostToolUse` : 도구 실행 완료 직후 자동 실행 (예: 파일 저장 즉시 `Prettier` / `ESLint` 자동 포매팅)

Module 12: 실전 Hooks 설정 예시

`.claude/hooks.json` 설정 파일

```
{
  "hooks": {
    "PreToolUse": [
      {
        "matcher": { "tool": "Bash", "command": "rm -rf *" },
        "action": "block",
        "message": "안전상의 이유로 광범위 삭제 명령은 금지되어 있습니다."
      }
    ],
    "PostToolUse": [
      {
        "matcher": { "tool": "Edit" },
        "action": "exec",
        "command": "npx prettier --write $CHANGED_FILE && npx eslint --fix $CHANGED_FILE"
      }
    ]
  }
}
```

효과: AI가 코드를 작성하는 족족 팀의 코딩 스타일로 자동 포매팅되어 코드 리뷰 피로도가 90% 이상 감소합니다!

Part 4. 종합 마무리 (Wrap-up & Quiz)

실전 베스트 프랙티스 & 자가진단 퀴즈

Module 13: 12개 핵심 개념 한눈에 정리

영역	핵심 개념	주요 목적 및 베스트 프랙티스
기본기	Agentic Loop	Context → Plan → Action → Verify 4단계 자율 사이클
워크플로우	EPCC 사이클	Explore와 Plan을 분리하여 성급한 코드 수정 방지
컨텍스트	<code>/compact</code> & <code>/clear</code>	Context Defocus 방지 및 토큰 비용 절감
설정 관리	<code>CLAUDE.md</code>	프로젝트 영구 메모리 (빌드/테스트 규칙, 컨벤션)
확장성	Subagents	독립 컨텍스트 기반 작업 위임 및 병렬 실행
온디맨드	Skills	<code>.claude/skills/</code> 필요할 때만 로드되는 절약형 지침
도구 확장	MCP Protocol	외부 DB, GitHub, API 연동 표준 규격
품질/안전	Hooks	Pre/Post Tool 생명주기를 통한 강제 포매팅 & 보안 가드레일

Module 13: 실전 자가진단 퀴즈 (1/2)

Q1. Claude Code와 일반적인 AI 챗봇의 가장 큰 차이점은?

1. 더 많은 한국어 단어를 알고 있다.
2. 로컬 파일시스템과 터미널에 직접 접근하여 파일 수정 및 명령어를 자율 실행한다.
3. 웹 브라우저에서만 동작한다.
4. 오직 파이썬 언어로만 코드를 작성할 수 있다.

(정답: 2번)

Q2. EPCC 워크플로우에서 복잡한 기능을 구현할 때 가장 먼저 해야 할 일은?

1. 즉시 코드를 작성하고 Git에 커밋한다.
2. 코드를 수정하지 않고 관련 코드베이스를 탐색(Explore)하고 계획(Plan)을 세운다.
3. 모든 의존성 패키지를 삭제하고 재설치한다.

(정답: 2번)

Module 13: 실전 자가진단 퀴즈 (2/2)

Q3. **CLAUDE.md**와 **Skills**의 차이점으로 가장 올바른 설명은?

1. **CLAUDE.md**는 항상 로드되고, **Skills**는 필요할 때만 온디맨드로 로드된다.
2. **Skills**는 삭제할 수 없는 시스템 파일이다.
3. **CLAUDE.md**에는 오직 작성자의 이름만 적어야 한다.

(정답: 1번)

Q4. 코드 수정 후 자동으로 Linter/Prettier를 실행하도록 강제하는 가장 적합한 도구는?

1. **/compact** 명령어
2. **PostToolUse** 훅(Hook)
3. MCP 서버

(정답: 2번)

 **수고하셨습니다!**

Claude Code와 함께 차세대 AI 페어 프로그래밍을 시작하세요.

생존코딩 오준석

- 질의응답 (Q&A)
- 실습 및 피드백 공유